Aaryan: handbook

Problem, domain, data and preprocessing

Read 00_Start_Here first; then this handbook.

Machine Learning (702DB0C015), B.Tech CSE (Data Science), Semester V, 2026-27

Contents

1. Part brief
2. Knowledge pack
3. Presentation guide
4. Viva questions and answers
5. Diagrams

Aaryan: problem, domain, data and preprocessing

Marks this part carries: Problem Understanding and Formulation (3), Data Preprocessing and Exploration (3, shared with Paras for the stamp-level preparation). **Why it matters:** if the data is wrong or leaks, every model and metric downstream is wrong. This is where the project learned its most expensive lessons (leakage, label and coordinate errors started here).

Your four jobs

1. **Problem formulation:** what we are solving, why it is hard, why difference images, why ML, how success is measured.
2. **The data:** where it comes from (ZTF via IRSA), what the four files are, how it was chosen and downloaded (metadata first, resumable, atomic), how nights were split, and why.
3. **Preprocessing and detection:** loading an exposure, bad-pixel masks, reference = science minus difference, background and noise, sky coordinates, and finding blobs in difference images (SEP, 2-sigma, minimum 5 pixels).
4. **Exploration and the leakage audit:** the threshold study, "most detections are noise", the stamp zoo, and the four kinds of leakage, with the experiment (100% -> 50% -> 99%) that proved it.

What is NOT yours

Labels, stamps and the CNN: Paras. Linking, trees, metrics: Satvik. Scoreboard, deep search, viewer: Advik. You hand Paras a clean table of blobs and a clean split.

Where your code is (in `project/`)

Area	Files	What they do
Acquire	`src/asthunt/acquire/query_ztf_metadata.py`, `query_ztf_field_night.py`, `select_night_exposures.py`, `load_unit_metadata.py`, `list_units.py`, `download_ztf_images.py`, `download_ztf_reference_image.py`, `find_quadrants_covering_position.py`	ask the archive; pick exposures; list units; download safely
Clean	`src/asthunt/clean/load_exposure.py`, `read_fits_image.py`, `read_exposure_header.py`, `build_bad_pixel_mask.py`, `estimate_background.py`, `load_night_images.py`	read the four files, build the mask, estimate background and noise
Detect	`src/asthunt/detect/extract_sources.py`	SEP matched-filter detection of positive blobs
Config	`src/asthunt/config/load_config.py`, `configs/acquire.yaml`, `detect.yaml`, `project.yaml`	all settings, including the **night split**

Area	Files	What they do
Leakage	`src/asthunt/evaluate/check_split_leakage.py`	lists groups that appear in two splits (must be empty)
Tests	`tests/acquire`, `tests/clean`, `tests/detect`, `tests/config`	all offline
Lessons and notebooks	`docs/project_setup.md`, `astronomy_data_primer.md`, `data_acquisition.md`, `difference_imaging.md`, `leakage_audit.md`, `kepler_post_check.md`; notebooks with the same names	

What you hand over

- The problem statement and related-work paragraph for the report (with the papers in `00_Start_Here/04_Course_Mapping.md`).
- Figures: the four-file picture, the noise histogram, the threshold table, the split diagram, the leakage experiment.
- A clean description of the data for the report (sources, size, split, preprocessing steps, what was masked and why).

Your seven key messages

1. **Difference images remove everything that did not change**, so only new or moving things remain.
2. **Ask before you download:** a metadata query (seconds) told us 1,552 exposures exist, 16.4% carry quality flags, and only 40 nights have at least 4 good frames.
3. **The split is by night, decided in a config file before modelling**, so it cannot change after disappointing results.
4. **Detection is tuned for completeness; classification for purity.** About 7,800 blobs per frame at 2 sigma, only 173 with SNR > 5, most are noise, and that is fine.
5. **The first dataset cheated** (four kinds of leakage); we proved it with an experiment and rebuilt the data.
6. **The test night is deliberately different** (sharper, deeper, near opposition): a distribution shift we want, and must read with care.
7. **Provenance:** you can say exactly which files and settings produced any result.

Numbers you must know by heart

Number	Meaning	Source
1,552	exposures of our quadrant in the archive table	`docs/data_acquisition.md`
898 / 478 / 176	r / g / i band exposures	same
16.4%	exposures with quality flags (we keep `infobits == 0`)	same

Number	Meaning	Source
8 / 2 / 1 / 1	train / validation / test / holdout nights	`configs/acquire.yaml`
220 files, 3.55 GB	first 11 nights x 5 frames + 1 reference	`docs/data_acquisition.md`
38 / 19 / 7 / 0.006 MB	science / mask / difference / PSF per exposure	same
3072 x 3080 px, 1.01 arcsec/px, 30 s	one quadrant, pixel scale, exposure	domain primer
1,950 -> 334	sources in science vs difference image at 5 sigma	`docs/difference_imaging.md`
7,764 blobs at 2 sigma; 173 with SNR > 5	detection counts on one frame	same
6.1k / 7.8k / 731 / 334	detections at 1.5 / 2 / 3 / 5 sigma	same
100% -> 50% -> 99%	leakage experiment	`docs/leakage_audit.md`
222.7 px vs 2.6 px	smallest random-bogus residual vs largest real residual	`docs/ml_basics.md`

Hands-on tasks (do these yourself)

1. `cd project && pytest tests/acquire tests/clean tests/detect tests/config -q`.
2. Re-run `notebooks/difference_imaging.ipynb`; produce the threshold table yourself; explain in two sentences why 1.5 sigma gives *fewer* detections than 2 sigma.
3. Re-run `notebooks/leakage_audit.ipynb`; reproduce 100% -> 50% -> 99%.
4. Open one mask file's header (`BIT00...BIT15`) and list which bits we treat as bad and why the ghost bit (5) is not.
5. Change `threshold_sigma` in a copy of `configs/detect.yaml` to 3.0; predict then check how the number of blobs changes.

Pitfalls

- Saying "we downloaded the data" without saying *how it was chosen* and *how it was split*.
- Confusing the **reference image** (old, deep) with the **science image** (tonight).
- Forgetting that `reference = science - difference` is computed (ZTF already aligned and blurred it).
- Mixing 0-based (numpy/SEP) and 1-based (FITS) pixel coordinates.
- Treating NaN as 0 without masking: its edge becomes a fake "detection".

Aaryan: knowledge pack

0. Read first (in this order)

`00_Start_Here/01_Project_Overview` -> `02_Domain_Primer` (all of it; you own the domain) -> `03_ML_Primer` (sections A, C, D) -> then the lessons `project/docs/project_setup.md`, `astronomy_data_primer.md`, `data_acquisition.md`, `difference_imaging.md`, `leakage_audit.md`, and the side check `kepler_post_check.md`.

---

1. Problem formulation (3 marks: write this section well)

The task. Given several consecutive 30-second exposures of the same patch of sky, find the objects that move across them (asteroids), decide which detections are real, and tell the known ones from the unknown ones. **Input:** a night of difference images of one CCD quadrant (5 frames, about 7,800 blobs each). **Output:** a ranked list of tracks with a score and a known/unknown flag. **Why it is hard:** (1) 97% of blobs are junk; (2) asteroids that matter are faint, barely above the noise; (3) there is no ground truth in real images (we must create labels); (4) the telescope pointing shifts between exposures; (5) public catalogues themselves have position errors. **Why ML:** junk comes in many shapes (dipoles, cosmic rays, edges, noise); a CNN learns them from examples where rules cannot; tree models combine many weak track clues; and the rest is physics (matched filter, linking, stacking), not learning. **Success is measured by** completeness (how many planted fakes are recovered, by brightness) at a fixed number of false alarms, **depth** (the brightness where 50% are recovered), known asteroids recovered, and unexplained alarms per night. Accuracy is *not* used (3% positives). **A baseline** is required: logistic regression on six numbers per blob (AP 0.780). Every later model must beat it. **Scope and honesty:** public data only; one field mostly; a research pipeline, not a discovery claim. **Origin:** a post claimed a planet candidate in old Kepler data found with AI-assisted code. We checked it (`kepler_post_check.md`): the 13.77-day signal matches the confirmed planet Kepler-169 e (chance of the period match about 1 in 350), simple blending cannot make a dip as deep as claimed (0.0685% maximum vs 0.24% reported), and the decisive test is whether the dips fall at that planet's transit times. We kept the habits: cross-match, quantify the coincidence, name the decisive test.

---

2. The data

2.1 Where it comes from

ZTF images, served by IRSA. For every exposure: science image (38 MB), mask (19 MB), difference image (7 MB, compressed), PSF (6 KB). The difference image is science minus a deep reference that ZTF has already resampled and blurred to match, so `reference = science − difference` gives us the reference on the same grid for free.

2.2 Metadata first

A single web query returned a table of 1,552 exposures of our quadrant (field 618, CCD 3, quadrant 4): 898 r-band, 478 g-band, 176 i-band; 16.4% had quality flags (`infobits != 0`), which we exclude. Choosing nights from this table costs seconds; downloading the wrong 3.5 GB costs an hour. Linking needs several exposures on the same night: only 40 nights had at least 4 good r-band frames.

2.3 The split (the most important design decision of the data stage)

split	nights	median seeing	median depth
train	8	1.86 px	20.32
validation	2	2.08 px	19.85
test	1 (2019-02-08)	1.60 px	21.08
A holdout night (2022-06-17) was added later for the scoreboard. The test night is the **best** night (sharp, deep, near opposition): the most promising for discovery, so it must not be tuned on. It is also a deliberate **distribution shift**: expect fainter sources than the model has seen much of.			
Why a config file: `configs/acquire.yaml` fixes the split before modelling, so it cannot drift after disappointing results. `check_split_leakage` must return an empty list for every pair of splits.			

2.4 The downloader (engineering rules you can explain)

- **Idempotent:** running it twice equals running it once (existing files are skipped).
- **Atomic:** a file is written as `.part` and renamed only when complete; a crash cannot leave a half file that looks finished.
- **Retry with growing waits** (20 s, 40 s, 60 s...) when the archive says 503 (busy); it did, twice, and recovery worked.
- **404 handling:** some exposures lack a product for good (e.g. the subtraction failed); they are recorded in `missing_exposures.csv` and skipped everywhere.
- **Total:** 11 nights x 5 frames + 1 reference = 220 files, 3.55 GB, nothing missing.
- **Provenance:** the metadata table is cached (`metadata__000618.csv`), so re-runs work offline and are repeatable even if the archive changes.

2.5 The mask

Each pixel has 16 yes/no flags. We mask: satellite track (0), bad responsivity (2, 3), noisy (4), spike (7), saturated (8), dead (9), NaN (10), halo (12). We do **not** mask the "ghost" bit (5): ghosts cover about 4% of the chip, masking them would hide real asteroids that cross them, and the model can learn what ghosts look like. We read the meaning of each bit from the mask file's own header (BIT00...BIT15) instead of guessing.

---

3. Preprocessing: what `load_exposure` does

1. Read the difference image, mask and PSF (and the science image if present).

2. Build the **bad-pixel mask**: ZTF flags + pixels near saturated stars (grown) + NaN pixels. About 6-12% of a difference image is NaN (regions that could not be subtracted); one example frame has 11.9% masked.
3. `reference = science − difference`.
4. Return images, mask, PSF (normalised to sum 1), header, time, zero point, gain. **Background and noise** (`estimate_background`): the sky level varies slowly (moonlight, clouds). SEP measures the sky in 64 x 64 boxes ignoring stars and interpolates a smooth map; the noise map is the scatter in each place. Robust noise = 1.4826 x MAD, because a bright blob must not inflate it (a plain standard deviation on our example frame was 15.1 vs 8.0 robust). **Sky coordinates (WCS):** pixel <-> RA/Dec. Pointing shifts between exposures; the "common grid" (Satvik's linking) depends on this.

---

4. Detection and exploration (EDA)

4.1 `extract_sources`

Mark NaN/bad pixels, subtract background, smooth with a star-shaped kernel (matched filter: random noise averages down, a star-shaped signal does not), keep groups of at least 5 connected pixels at least 2 sigma above the noise, measure flux in a 3-pixel circle, SNR = flux / flux error. Only **positive** blobs are kept: an asteroid is in tonight's image, not in the reference.

4.2 The threshold surprise (one training frame)

threshold	detections	median size
1.5 sigma	6,141	11 px
2.0 sigma	**7,764**	6 px
3.0 sigma	731	8 px
5.0 sigma	334	9 px
At 1.5 sigma neighbouring noise peaks touch and **merge** into big blobs, so there are *fewer* detections and a faint asteroid next to a noise blob gets swallowed. A rule of thumb is a start; measuring is better. 2 sigma was chosen and later checked with injections.		

4.3 Most detections are noise, and that is fine

At 2 sigma about 7,800 detections per frame, only 173 with SNR > 5 (see `07_Diagrams/03_preprocessing_funnel.svg`). Detection is tuned for **completeness** (miss nothing); the CNN (Paras) is tuned for **purity**. Science image: 1,950 sources at 5 sigma; difference image: 334 (about 6x fewer).

4.4 The stamp zoo (what the three panels show)

See Paras's `triplet_stamp.svg`: a real object appears in science and difference but not in reference; a bad subtraction shows a star in both and a dipole in the difference; a cosmic ray is a sharp spike narrower than a star; noise is a faint speck; a masked edge is a step.

4.5 The combinatorics problem

5 frames x about 8,000 blobs: one blob per frame in every combination is 8,000^5 = about 3 x 10^19 possible tracks. Two fixes later in the pipeline: the classifier removes most junk first, and the linker only tries combinations consistent with straight-line motion.

---

5. The leakage audit (your signature story)

Our very first dataset (`legacy/tracklets.csv` , 3,530 rows, 50% real) gave 99.8% accuracy, then 100% with log features. A perfect score is a symptom; we asked what would have to be true of the data.

Kind	What it is	Ours
Target leakage	a feature contains the answer	`mag` , `true_rate` , `true_angle` filled for every planted fake and for no bogus row
Group leakage	train and test share a source	all tracklets came from the same 4 images
Shortcut learning	classes differ in an easy unreal way	random bogus tracklets had residual >= 222.7 px; real ones <= 2.6 px, so one rule separates them
Distribution shift	deployment differs from training	test night sharper and deeper
The experiment: trained on easy data and tested on easy data: **100%**; trained on easy data, tested on realistic near-miss tracklets: **50%** (a coin flip); trained on realistic data (real + near misses), tested on realistic: **99%**.		
Hard negatives teach most. By how far one detection is moved off the line: 3-10 px -> 99%; 1-3 px -> 96%; 0.5-1.5 px -> 89% (a source 1 px off a line is nearly invisible to position-only features, which is why Paras's CNN looks at pixels).		
Rules that came out of it: features must be measurable on new data; split by night; derived examples go with their source; negatives must be realistic.		

---

6. Project hygiene (Project setup lesson; two sentences each)

- **Environment:** `environment.yml` rebuilds the same software with one command; the laptop had at least 6 Pythons, so "which Python ran this" was unknowable before.
- **Layout:** `src/asthunt/<stage>/<verb_object>.py` , one purpose per file; settings in `configs/*.yaml` ; tests offline; old scripts kept in `legacy/` unchanged and rebuilt properly.

---

7. Cross-part cheat sheet

- **Paras:** labels from planted fakes, known asteroids (Horizons) and "the rest"; 3x31x31 stamps normalised by MAD; CNN with 72,145 weights; AP 0.899 vs 0.780; cross-fitting.
- **Satvik:** linking by seed-and-predict with KD-trees on the common sky grid (minimum 8 arcsec/hour, at least 3 of 5 frames); 17 features; XGBoost; metrics (accuracy paradox, PR vs ROC); thresholds chosen on validation.
- **Advik:** one deployed path and bundles; evaluation on planted fakes and held-out nights; depth 20.5 -> 21.4 -> 22.1; deep search by shift-and-stack with a 6-sigma cut; the live viewer.

8. Self-quiz (answers in `04_Viva_QA.md`)

1. Why query metadata before downloading? 2. Why must the split live in a config written before modelling? 3. What two properties make the downloader safe to re-run? 4. Why is the ghost bit not masked? 5. The test night is sharper and deeper than training: is that a problem? 6. Why do we keep only positive blobs? 7. Why did 1.5 sigma give fewer detections than 2 sigma? 8. Why is it acceptable that most detections are noise? 9. Name the four leakage types with one example each. 10. What did the 100% -> 50% -> 99% experiment prove?

Aaryan: presentation guide

Your slot: 4 minutes, 4 slides. (In the 12-minute plan: 2 minutes, slides 1 and 4 plus one sentence on the split.) Pictures live in `07_Diagrams/` and `../00_Start_Here/11_Diagrams/`.

Slide 1: "The data and the problem" (1 min)

On the slide: `data_flow.svg`; three bullets: ZTF difference images; 5 frames of 30 s each night; 97% of blobs are junk. **Say:** "The data is public, from the Zwicky Transient Facility at Palomar, served by NASA's IRSA archive. For every 30-second exposure we get four files: the science image, a mask of bad pixels, a difference image and the blur shape. Before downloading anything we asked the archive for its table of exposures: 1,552 for our patch of sky, 16% with quality flags which we drop. Then we chose nights, wrote the split into a config file, and downloaded safely: the downloader can be re-run, writes files only when complete, and waits and retries when the archive is busy, which it was, twice."

Slide 2: "Difference imaging" (1 min)

On the slide: `difference_imaging.svg`. **Say:** "A telescope picture is mostly stars. We use ZTF's difference image: tonight's picture minus a deep picture of the same sky from long ago. Everything that did not change cancels. What is left is new or moving things, which is what we want, but also leftovers from imperfect subtraction: dipoles from bright stars, cosmic rays, noise. Most of what remains is junk, which is why we need a classifier. Detection itself is simple: smooth with the shape of a star and mark everything two times the noise or more. That gives about 7,800 blobs per frame, only 173 of them strong. We tuned detection to miss nothing, and leave the junk removal to the neural network."

Slide 3: "Splitting by night and the leakage audit" (1 min 15 s)

On the slide: `../00_Start_Here/11_Diagrams/02_split_by_night.svg` on the left and `07_Diagrams/04_leakage_types.svg` (or only its red experiment box) on the right. **Say:** "We split by night, not by random rows, and wrote the split into a config before any modelling: eight nights to train, two to validate, one best night to test, and one never-touched holdout. Frames from one night share stars, noise and artifacts, so a random split would leak. This mattered, because our very first dataset cheated in four ways: columns that existed only for fake asteroids, all samples from the same four images, bogus examples so easy that one rule separated them, and a test night that was different. We proved it: trained and tested on the easy data the model scored 100%; trained on the easy data and tested on realistic near-misses it scored 50%, a coin flip; trained on realistic data it scored 99%."

Slide 4: "What we hand to the model" (45 s)

On the slide: `preprocessing_funnel.svg`. **Say:** "After this stage the next person receives clean exposures, a mask, a noise estimate, and a table of blobs on a split that cannot leak. The numbers to remember: 8, 2, 1, 1 nights; about 7,800 blobs per frame; 3% of them real. Over to Paras, who will turn each blob into three small pictures and teach a neural network to judge them."

Delivery tips

- Keep the *why* before the *what*: say why you ask for metadata first, why split by night.
- On slide 3, slow down at 100% -> 50% -> 99%: it is the strongest evidence in your part.
- Do not read numbers from the slide; say them naturally and round.
- If asked about astronomy details you do not know, say what you do know (the idea) and offer the term from the glossary.

Anticipated first questions

1. "Why a difference image?" -> static things cancel; only new or moving remain. 2. "Why split by night?" -> shared stars/noise/artifacts would leak. 3. "Why 2 sigma?" -> tuned for completeness, measured, later checked with injections. 4. "What is a mask?" -> per-pixel flags; we mask defects but not ghosts. 5. "How do you know the data is good?" -> quality flags dropped, bad pixels masked, split audited.

Aaryan: viva questions and model answers

A. Problem formulation

1. What problem does the project solve? Finding moving objects (asteroids) in consecutive telescope images, deciding which detections are real, and separating known from unknown ones, with an honest measurement of how deep and how reliable the search is.

2. What are the input and output of the whole system? Input: a night of difference images for one CCD quadrant (5 frames). Output: a ranked list of tracks with a score and a known/unknown flag, for a human to examine.

3. Why is the problem hard? Only about 3% of blobs are real; the interesting asteroids are barely above the noise; real images have no ground truth; the telescope pointing shifts between exposures; and the catalogues we compare with have position errors.

4. Why use machine learning? Junk blobs have many shapes that rules cannot capture, so a CNN learns them from examples; trees combine many weak track-level clues. The rest (matched filter, linking, shift-and-stack) is physics and search.

5. How do you measure success? Completeness (fraction of planted fakes recovered) by brightness, depth (50% completeness magnitude), known asteroids recovered, unexplained alarms per night, and for classifiers recall at a fixed false-alarm level and average precision. Not accuracy.

6. Which paper does the project relate to? The closest is Duev et al. 2019 (real-bogus classification for ZTF with a CNN); also Mahabal et al. 2019 (ML for ZTF), Whidden et al. 2019 (shift-and-stack on GPUs), Masci et al. 2019 (ZTF data processing). Say what we replicated (a CNN on science/reference/difference triplets) and what we changed (31x31 stamps, our own labels).

7. What was the origin of the project and what did you check? A post that claimed a planet candidate in old Kepler data. We checked it: the period matches a confirmed planet (Kepler-169 e, 13.7671 days; chance about 1 in 350), simple blending cannot make a dip as deep as claimed, and the decisive test is whether the dips fall at that planet's transit times.

B. Data

8. Where does the data come from and what is in it? ZTF images from the IRSA archive. Per exposure: science image (38 MB), mask (19 MB), difference image (7 MB), PSF model (6 KB). A "unit" is field + CCD + quadrant + night.

9. Why query the metadata before downloading? It takes seconds, shows what exists (1,552 exposures, 898 r-band, 16.4% flagged, 40 usable nights), and lets us choose data on purpose; downloading the wrong 3.5 GB costs an hour.

10. How was the split decided and why by night? 8 train, 2 validation, 1 test (2019-02-08), 1 holdout (2022-06-17), written in `configs/acquire.yaml` before modelling. Frames of one night share stars, noise

and artifacts; random splits would leak that.

11. Is it a problem that the test night is better than the training nights? It is a deliberate distribution shift: the test night is the sharpest and deepest, near opposition, so the most promising for discovery. We expect fainter sources than the model saw much of and read its scores with that in mind.

12. What makes your downloader safe to re-run after a crash? It is idempotent (existing files skipped) and atomic (written as `.part`, renamed when complete). It also retries with growing waits and records exposures that lack a product.

13. What is a unit and why are results per unit? One field + CCD + quadrant + night, e.g. `000618_c03_q4_2019-02-08`. Linking needs frames of the same sky on the same night, and each quadrant has its own pixel grid and metadata.

C. Preprocessing and detection

14. What are the four files and what does the mask do? Science (tonight), difference (science minus aligned reference), mask (16 flags per pixel: saturated, dead, cosmic ray, ghost...), PSF (blur shape). We mask defects but not the ghost bit, because ghosts cover about 4% of the chip and masking them would hide real asteroids.

15. How do you get the reference image? `reference = science − difference`. ZTF already resampled and blurred the reference to match the science image, so no reprojection code is needed.

16. How is noise estimated and why robustly? SEP estimates the sky level and scatter in 64x64 boxes ignoring stars; for single images we use sigma = 1.4826 x MAD. Bright blobs inflate a plain standard deviation (15.1 vs 8.0 on an example frame).

17. Why keep only positive blobs? An asteroid is present tonight and absent in the reference, so it is a positive blob; negative blobs are things that faded or moved away.

18. Why did 1.5 sigma give fewer detections than 2 sigma? At the lower threshold neighbouring noise peaks touch and merge into large blobs, so the count drops (6,141 vs 7,764), and a faint asteroid next to a noise blob gets swallowed.

19. 7,800 blobs per frame, 173 with SNR > 5. Is that acceptable? Yes: detection is tuned for completeness (do not miss real things), classification for purity (throw junk away). Two steps, each doing what it is good at.

20. What is the combinatorics problem? 5 frames x about 8,000 blobs: all one-per-frame combinations are 8,000^5 = about 3 x 10^19. The CNN removes most junk first and the linker only tries combinations consistent with straight-line motion.

21. What is the WCS and why does it matter? The mapping between pixels and sky coordinates. Pointing shifts up to 12 px between exposures, so asteroids move in straight lines on the sky, not in each frame's pixels; linking must use sky-based common coordinates.

D. Leakage and exploration

22. Name the four kinds of leakage with an example each. Target (`mag` existed only for fakes), group (all tracklets from 4 images), shortcut (random bogus tracklets >= 222 px off a line), distribution shift (test night sharper/deeper).

23. What did the 100% -> 50% -> 99% experiment show? A model trained on easy data scored 100% on easy test data but 50% (a coin flip) on realistic near-miss tracklets; trained on realistic data it scored 99%. A perfect score was a symptom of a too-easy dataset, not of a good model.

24. What are hard negatives and why do they help? Negatives that look like the positives (here, near misses a few pixels off a line). The model learns most from them: accuracy was 99% with offsets of 3-10 px, 96% at 1-3 px and 89% at 0.5-1.5 px, which also shows how hard such cases are for position-only features.

25. How do you check that no night appears in two splits? `check_split_leakage(a, b)` lists groups appearing on both sides; it must return an empty list for train/validation, train/test and validation/test.

26. What would you do if a result looked too good? Ask what would have to be true of the data for it to be so easy; check features for target leakage, splits for shared sources, negatives for realism, and test on a different kind of data.

E. General ML and hygiene

27. What is overfitting and how does the split protect against it? Overfitting is memorising training noise. Only data the model has never seen shows whether it learned a general rule, so settings are chosen on validation and the test is opened once.

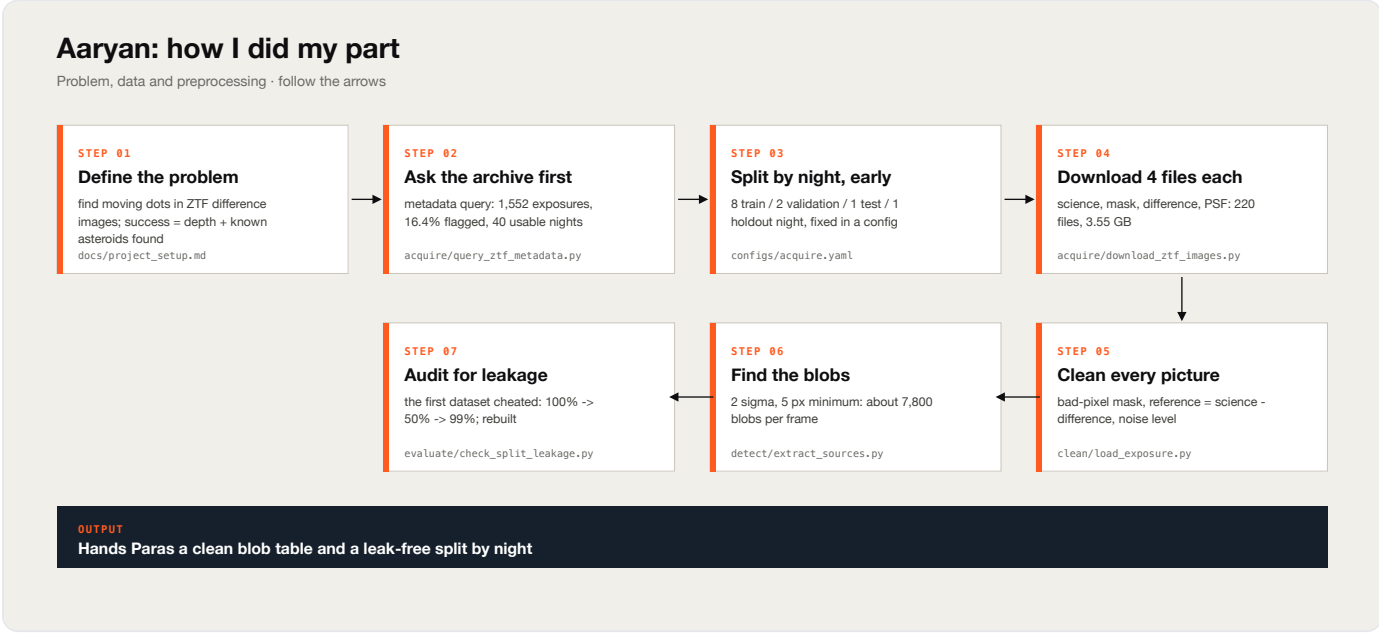
28. Why is accuracy a poor metric here? 3% positives: "always bogus" gets 96.8% accuracy with zero recall. Use recall, precision and average precision.

29. What does `environment.yml` give you? One command rebuilds the same software; without it "it worked on my machine" is unprovable (the laptop had at least 6 different Pythons).

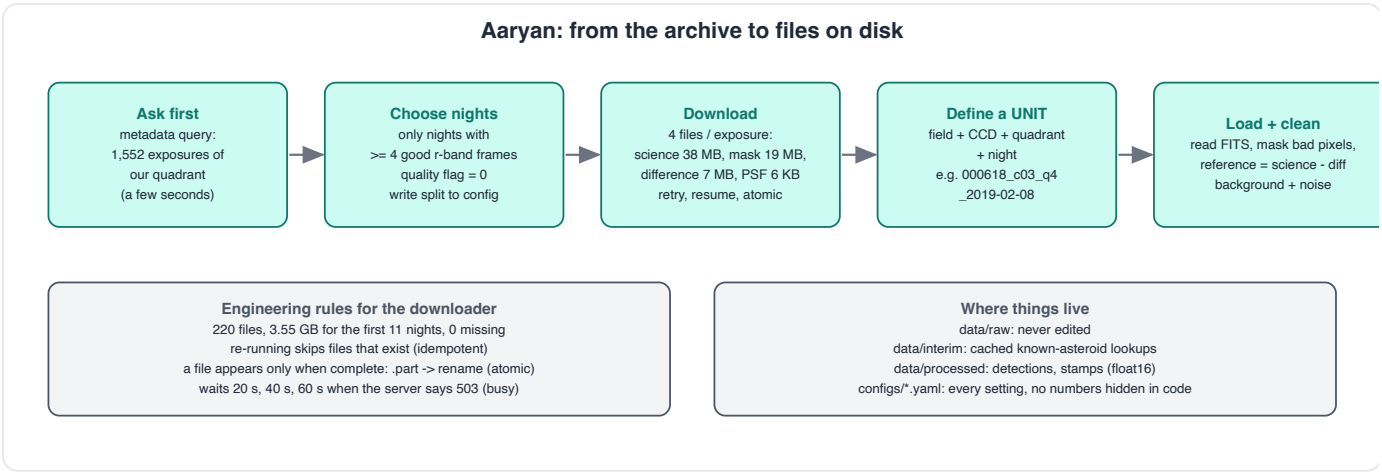
30. What would you improve in the data stage? More fields and nights, especially near opposition; a formal learning-curve study (how performance grows with the number of nights); and an automatic check that archive quality flags and our mask agree.

31. "Did you write this code yourself?" Answer truthfully (see `00_Start_Here/07_AI_Use_and_Integrity.md`): the project was developed with an AI coding assistant under Advik's direction; say what you re-ran, changed or extended in the data stage, then explain a step (for example the threshold table) without notes.

Diagrams



00 Process Diagram Aaryan



01 data flow

Aaryan: difference imaging, why only new things remain



What a difference image contains

new or moving thing	star-shaped POSITIVE blob (what we want)
faded or departed thing	NEGATIVE blob (we ignore these)
bright star, imperfect subtraction	dipole: a bright half and a dark half side by side
cosmic ray	sharp spike, narrower than a star
pure noise	random speckle; 2-sigma blips pass the threshold by chance

02 difference imaging

Aaryan: how many 'things' are left at each step (one training frame)

1,950	sources in the SCIENCE image
334	sources in the DIFFERENCE image at 5-sigma
7,764	blobs at 2-sigma (lenient on purpose: do not miss anything)
173	of those have SNR > 5
~1,000	real detections per night (planted fakes + known asteroids) among ~36,000 blobs

Detection is tuned for COMPLETENESS (miss nothing). Classification, next, is tuned for PURITY (throw junk away).

03 preprocessing funnel

Aaryan: four ways a model can cheat, and what we found in our own first data

1. Target leakage	2. Group leakage	3. Shortcut learning	4. Distribution shift
a feature contains the answer	train and test share a source	classes differ in an easy, unreal way	deployment data differs from training
ours mag, true_rate existed only for rows WE made: 'is mag missing?' = the label	**ours** all tracklets came from the same 4 images	**ours** random bogus tracklets were >= 222 px off a line; real ones <= 2.6 px	**ours** test night is sharper and deeper than training nights
rule feature must be measurable on new data	**rule** split by NIGHT; check_split_leakage must be empty	**rule** build hard negatives (near misses)	**rule** expect and measure it; keep a holdout

The experiment that proved it (same model, different test data)

trained on easy legacy data -> tested on easy legacy data	**100%**
trained on easy legacy data -> tested on REAL near misses	**50% (a coin flip)**
trained on real + near misses -> tested on real + near misses	**99%**

04 leakage types